Deep Learning

Module 2: Training Neural Networks
Data Visualization
Tensorflow

Soumia Chandran M

Syllabus – Module 2

Training Neural Networks

Initialization, dropout, batch normalization, overfitting, underfitting, training and validation curves.

Data Visualization

Feature and weight visualization, t-SNE.

Introduction to TensorFlow

Graphs, nodes, Tensor data structures – rank, shape, type; building neural networks with TensorFlow; introduction to Keras.

Training Neural Networks: Initialization

The common training process for a neural network is:

1. Initialize the parameters
2. Choose an optimization algorithm
3. Repeat the following steps:
   - Forward propagate an input
   - Compute the cost function
   - Compute the gradients of the cost with respect to the parameters using backpropagation
   - Update each parameter using the gradients, according to the optimization algorithm

Given a new data point, the trained model can then be used to predict its class.

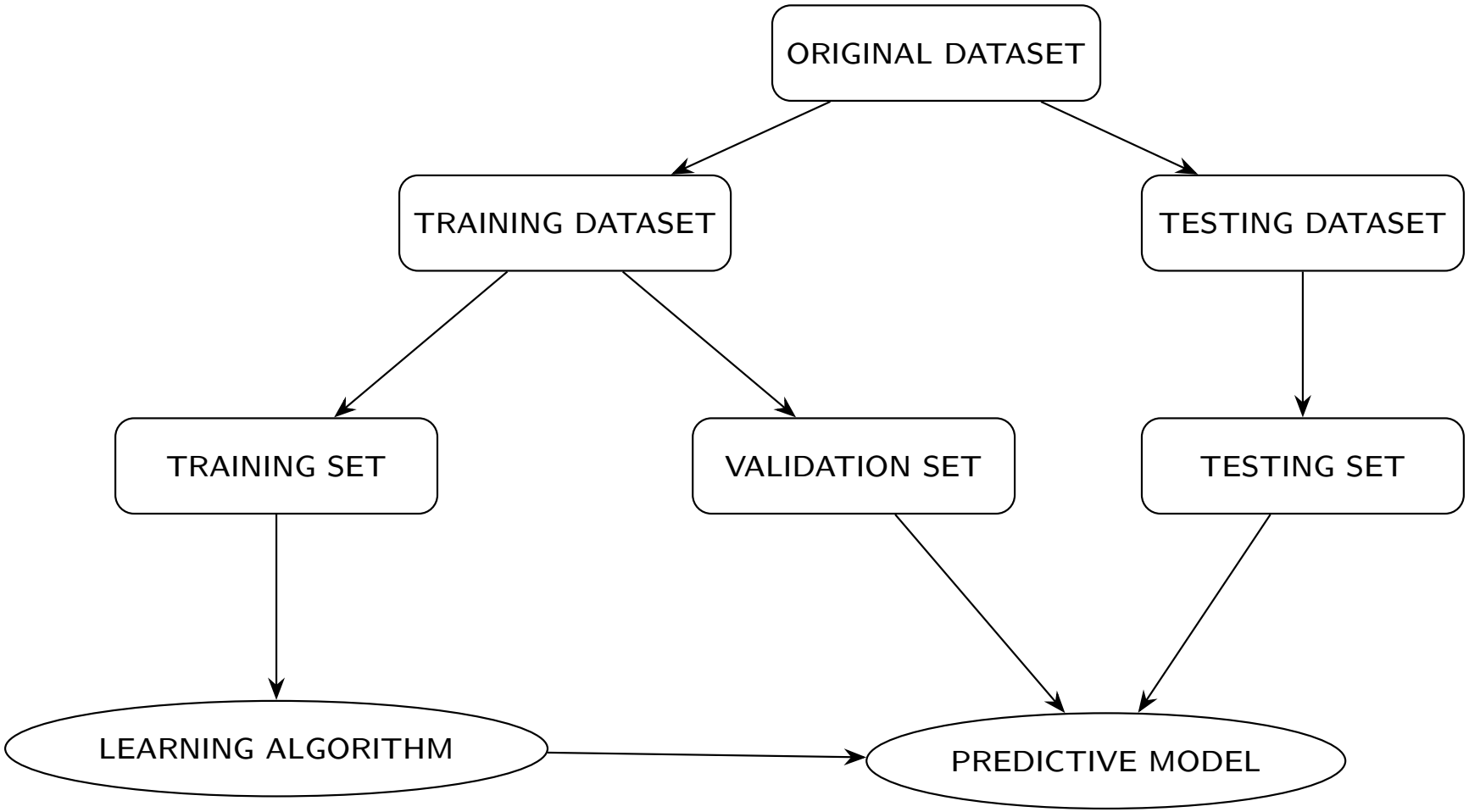
Hyperparameters

- A deep learning model has variable parameters that must be set **before** training – these are called **hyperparameters**.
- Hyperparameters affect the results of the model, so optimal values must be found for them.
- Finding the most optimal combination of hyperparameters is called **hyperparameter tuning**.
- Hyperparameter tuning is the process of finding the optimal values for a network's hyperparameters; it is done before training begins.

Dataset

- A dataset contains the information used to create a model.
- It is a collection of data structured as a table of rows and columns, arranged in some order.
- Dataset types:
  1. **Numerical** dataset (e.g. temperature, prices)
  2. **Categorical** dataset (e.g. Y/N, T/F)
  3. **Ordinal** dataset (similar to categorical, but ordered)
- A dataset can be represented as a table, a matrix, or a series of arrays.
- Data is usually stored in a data file (CSV, Excel, etc.).
- For an ML/DL application, a dataset is divided into:
  1. Training dataset
  2. Testing dataset

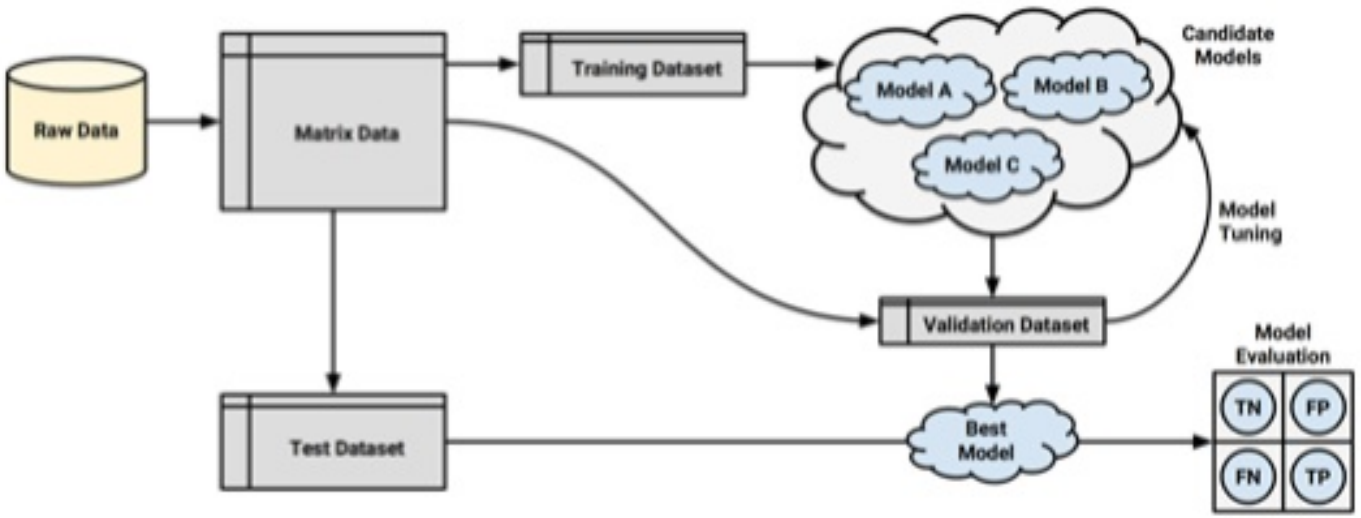
Dataset Split: Overview



Training and Testing

- **Training data**, — is the information used to train an algorithm.
- The training data includes both input data and the corresponding expected output.
- Based on this “ground truth” data, the algorithm can learn how to apply technologies such as neural networks, to learn and produce complex results, so that it can make accurate decisions when later presented with new data.
- Training data can be used for various machine learning algorithms, such as sentiment analysis, natural language processing, and chatbot training.
- **Testing data**, on the other hand, includes only input data, not the corresponding expected output.
- The testing data is used to assess how well your algorithm was trained, and to estimate model properties.

Machine Learning Implementation



Training Set

- A **subset** of the original dataset, used to learn the data's behavior (i.e., the model).
- The model learns and repeatedly evaluates the data to understand its behavior, then adjusts itself.
- Training data is used to **fit** the model – it trains on images and labels.
- Weights and biases are adjusted by looking at the training data.
- Accuracy is checked and reported after each iteration (a **batch**); the goal is to drive the error to zero.
- In each batch, weights and biases are adjusted, accuracy is checked, and the process repeats until the predicted output is correct.

Validation Set

- Used to evaluate the model **during** training.
- Accuracy is checked and reported after each **epoch**.
- The model does **not** update any weight or bias here.
- The ML/DL engineer monitors the validation error and tunes hyperparameters to optimize performance.

Testing Set

- Used to evaluate the **final** trained model.
- Checks accuracy after training is complete.
- Provides a final, real-world check on an unseen dataset to confirm the model was trained effectively.

Bias

- Bias is the error introduced by approximating a real-world problem, which may be complex, with a simplified model.
- High bias can cause an algorithm to miss relevant relations between features and outputs (**underfitting**).

Bias – Diagram

What is bias?

- Bias is the difference between the average prediction of our model and the correct value which we are trying to predict.
- Model with high bias pays very little attention to the training data and oversimplifies the model.
- It always leads to high error on training and test data.

Variance

- **Variance** is the variability of a model's predictions for a given data point – it reflects the spread of the model's output.
- A model with high variance pays too much attention to the training data and does not generalize to unseen data.
- Such models perform very well on training data but have high error rates on test data (**overfitting**).

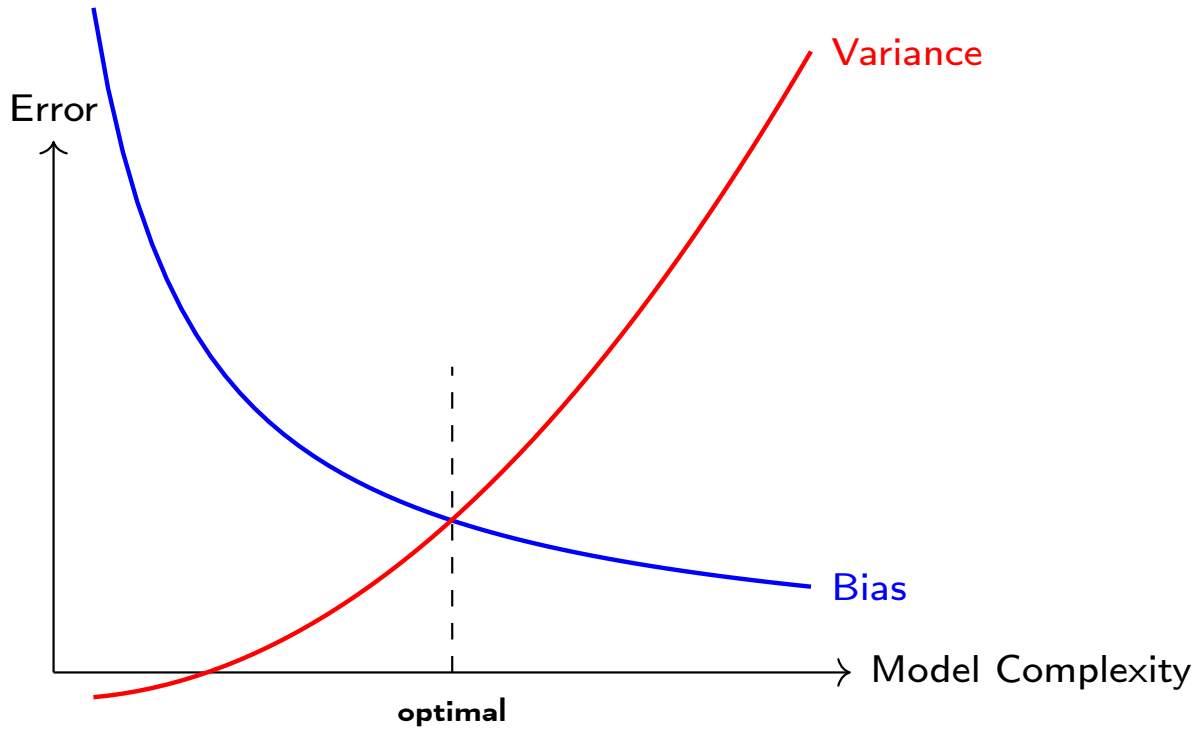
Bias and Variance – Target Diagram

Bias-Variance Tradeoff

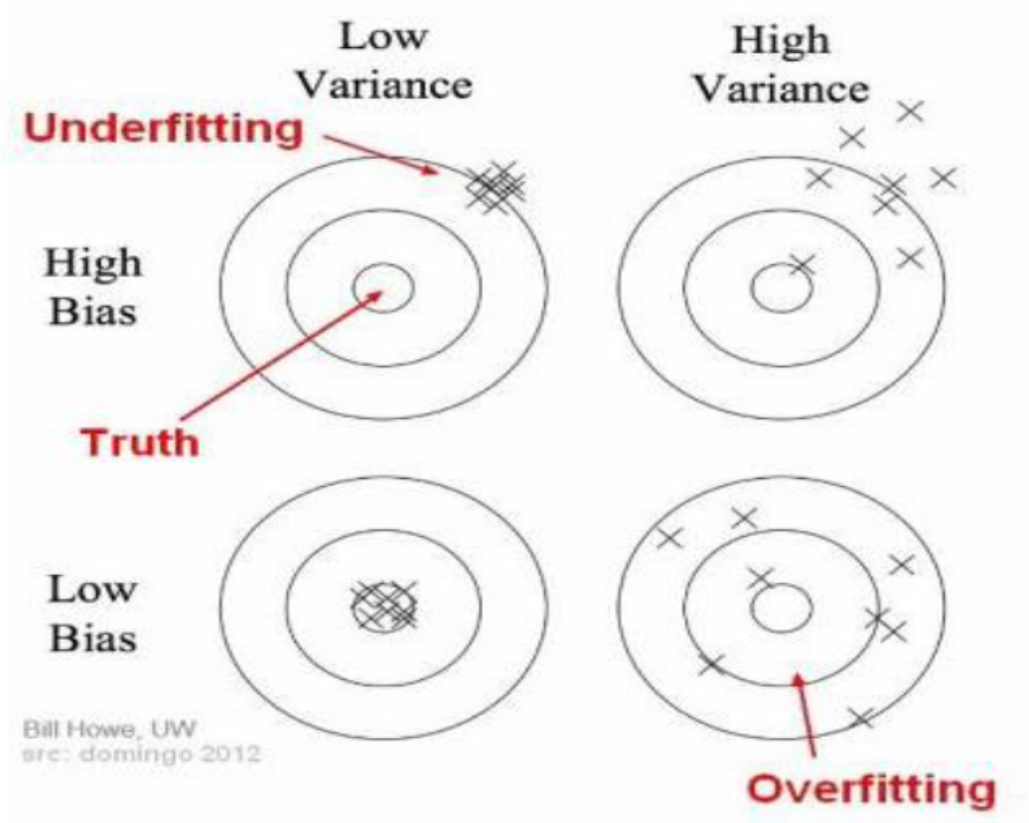
- Let's say we have a model which is very accurate, therefore the error of our model will be low, meaning a low bias and low variance.
- All the data points fit within the bulls-eye.
- Similarly we can say that if the variance increases, the spread of our data point increases which results in less accurate prediction.
- And as the bias increases the error between our predicted value and the observed values increases.

Bias–Variance Tradeoff

- To attain an accurate model, we seek to reduce both bias and variance.
- Bias and variance are at odds with each other: reducing one tends to increase the other. This is the **bias–variance tradeoff**.



Bias–Variance Tradeoff – Original Diagram



Overfitting and underfitting

- In supervised learning, **underfitting** happens when a model unable to capture the underlying pattern of the data.
- These models usually have high bias and low variance.
- It happens when we have very less amount of data to build an accurate model or when we try to build a linear model with a nonlinear data.

Training vs. Test Error (contd.)

- In supervised learning, **overfitting** happens when our model captures the noise along with the underlying patterns in data.
- It happens when we train our model a lot over noisy dataset.
- These models have low bias and high variance.
- These models are very complex like Decision trees which are prone to overfitting.

Overfitting

- As the number of parameters grows, networks gain the freedom to fit many types of datasets – which is what makes them powerful.
- Sometimes this power becomes a weakness: the network tries to memorize each data point, performing well on training data but poorly on test data. This is **overfitting**.
- Overfitting occurs when a model fits very noisy data too closely, so the trend no longer reflects reality.
- It is spotted when training error decreases to a very small value while error on new/test data increases.
- A common cause is a **small training dataset**: with little data, the network gains excessive control and satisfies every data point exactly, rather than capturing the general trend.

Overfitting in a Neural Network

- Occurs when the network learns too many details (and the noise) present in the training data.
- This results in poor performance on unseen/test data.
- The network fails to **generalize** the patterns found in the training data.
- If training error is low but test error is high, this signals overfitting.

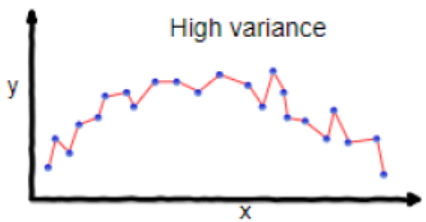
Underfitting

- A model is said to **underfit** when it cannot capture the underlying trend of the data (like trying to fit undersized pants!).
- Underfitting destroys the accuracy of the model – the model/algorithm does not fit the data well enough.
- Usually happens with too little data, or when a linear model is used on non-linear data.
- In short: Underfitting $\Rightarrow$ **high bias, low variance**.
- Detected when training accuracy itself is low / training loss is high.

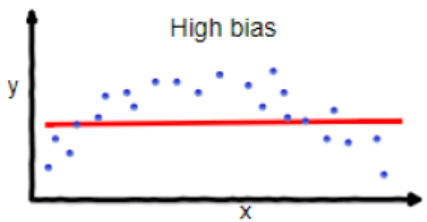
Techniques to reduce underfitting

1. Increase model complexity
2. Increase the number of features (feature engineering)
3. Remove noise from the data
4. Increase the number of epochs / training duration

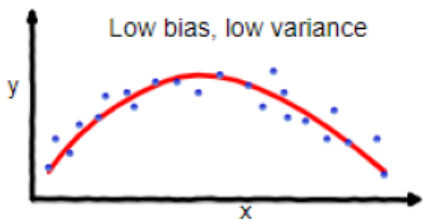
Overfitting vs. Underfitting – Diagram



overfitting



underfitting



Good balance

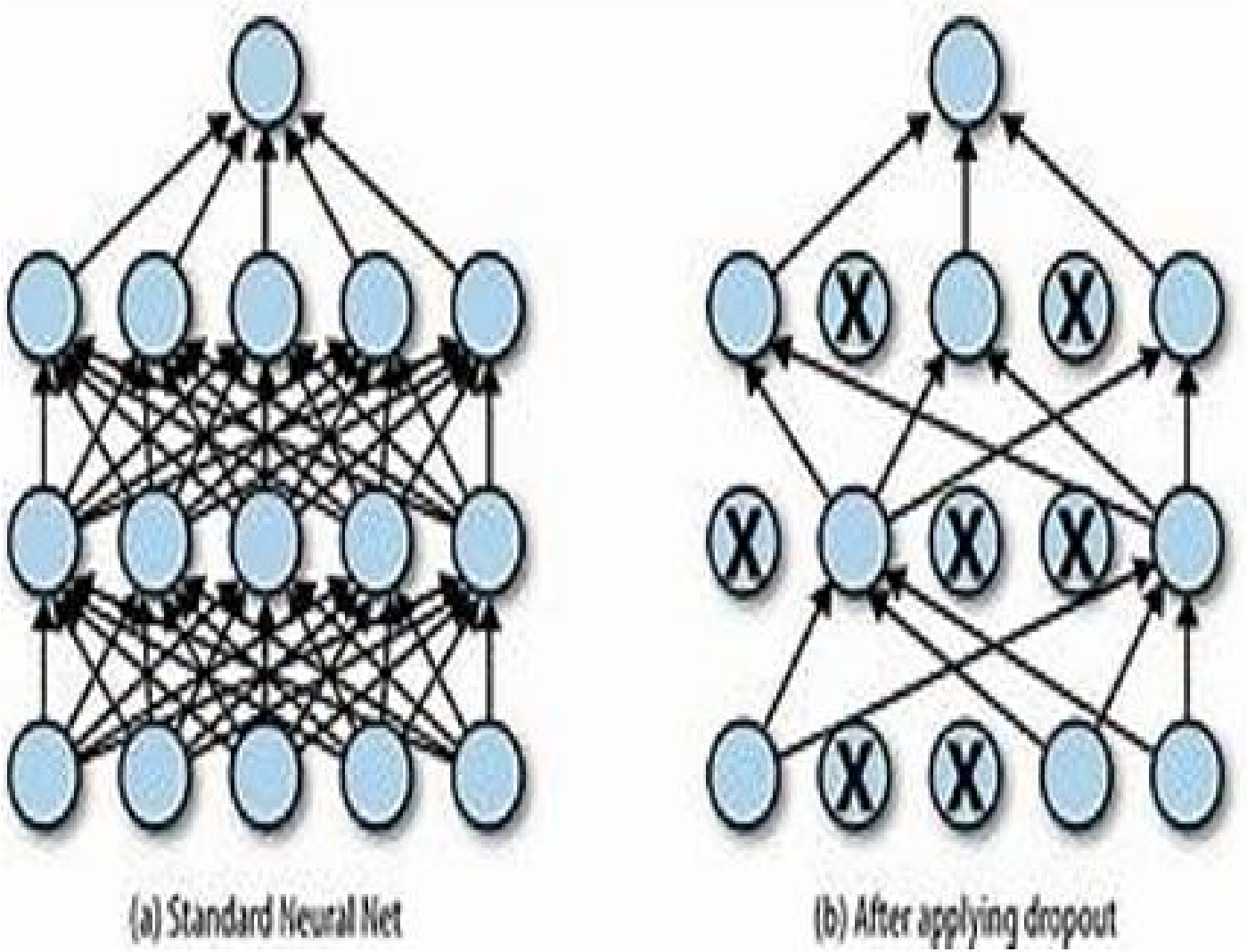
Model Flexibility: Polynomial Degree

- The degree of a polynomial model represents its flexibility: a higher degree lets the model fit more data points exactly.
- Constant (deg. 0) $\rightarrow$ Linear (deg. 1) $\rightarrow$ Quadratic (deg. 2) $\rightarrow$ Cubic, Quartic, Quintic, ...
- A **low degree** tends to **underfit**; a **very high degree** tends to **overfit** by chasing noise.

Dropout Layer

- One of the most popular **regularization** techniques to reduce overfitting in deep learning models.
- Overfitting occurs when a model shows high accuracy on training data but low accuracy on unseen/test data.
- Dropout was one of the first "deep" regularization techniques.
- It consists of removing units at random during the forward pass on each training sample, and putting them all back at test time.
- Introduced by **Geoffrey Hinton in 2012**.

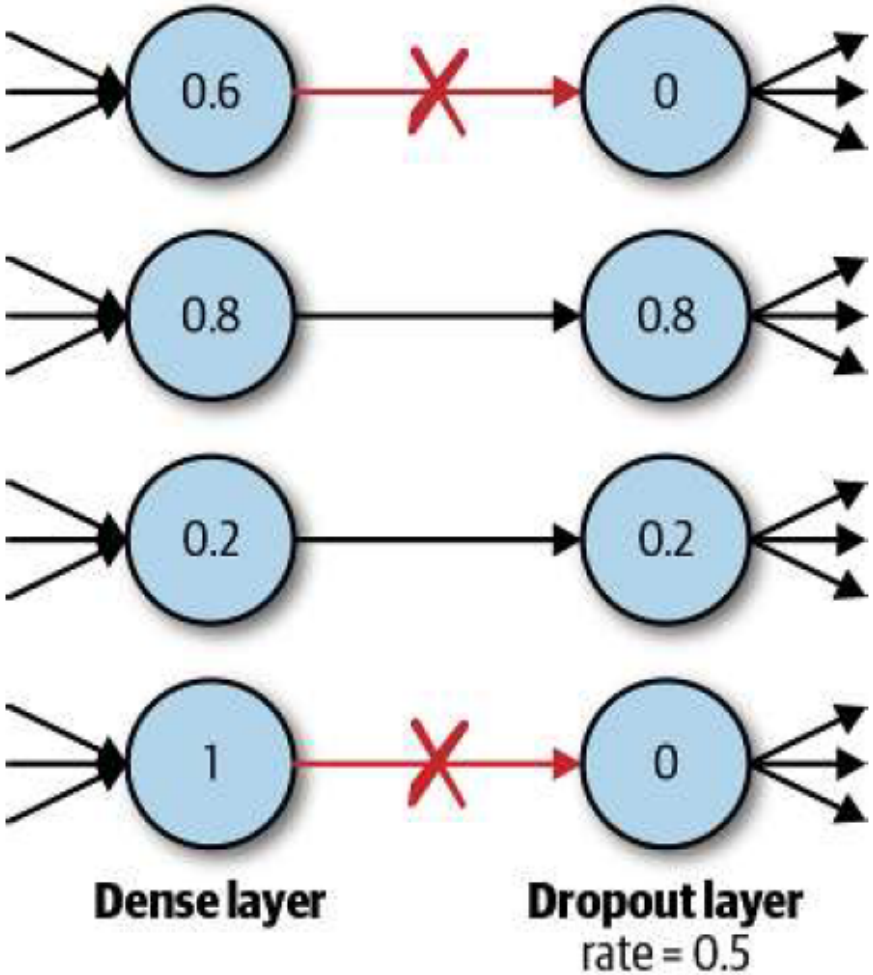
Dropout – Illustration



How Dropout Works

- During training, each dropout layer chooses a random set of units from the preceding layer and **sets their output to zero**.
- Units to drop are selected independently, at random, for every sample.
- The backward pass runs only through the kept units.
- At test time, **no** units are dropped – the full network is used for predictions.
- There are **no weights to learn** within a dropout layer, since dropped units are chosen stochastically.

Dropout Layer – Network View



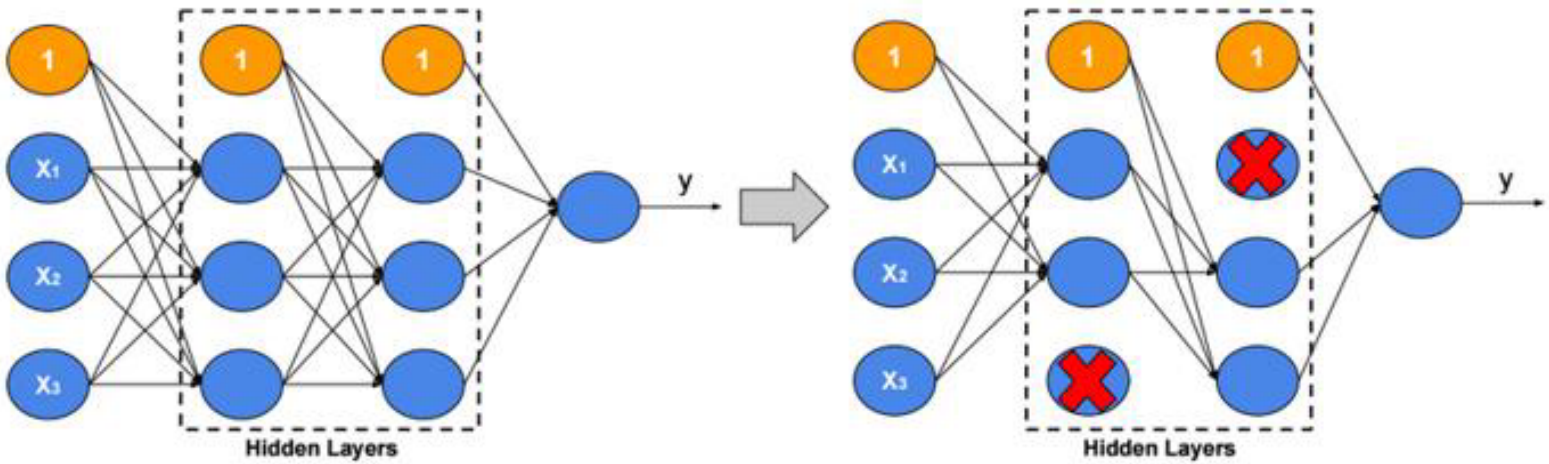
Dropout in Practice

- In Keras: `Dropout(rate = 0.25)` – `rate` is the proportion of units dropped from the preceding layer.
- In PyTorch: `nn.Dropout`, default probability $p = 0.5$.
- Used most often after **Dense** layers (most prone to overfitting due to their high weight count), though also usable after convolutional layers.
- **Inverted dropout** (the standard variant): multiplies activations by $\frac{1}{1-p}$ during training and leaves the network untouched at test time.
- Batch normalization has also been shown to reduce overfitting; many modern architectures use it **instead of** dropout.

Why Dropout Helps

- Drastically reduces overfitting by preventing the network from becoming **overdependent** on specific units or groups of units.
- The network cannot rely too heavily on any single unit, so knowledge is spread more evenly across the network.
- This improves generalization: the network is trained to make accurate predictions even under the "unfamiliar" conditions caused by dropped units.

Dropout – Backward Pass through Kept Units



Why Normalize?

- In a deep neural network, **internal covariate shift** is the change in the distribution of inputs to a layer, caused by ever-changing parameters during training.
- Input features with large numerical ranges can dominate training and bias the network toward only those features.

Example

Feature 1 ranges over $[1, 5]$; feature 2 ranges over $[100, 10000]$. During training, feature 2 dominates the network due to its larger scale.

This motivated the concept of **normalization**.

Benefits of Normalization

Normalization rescales numeric features to a similar range. Advantages include:

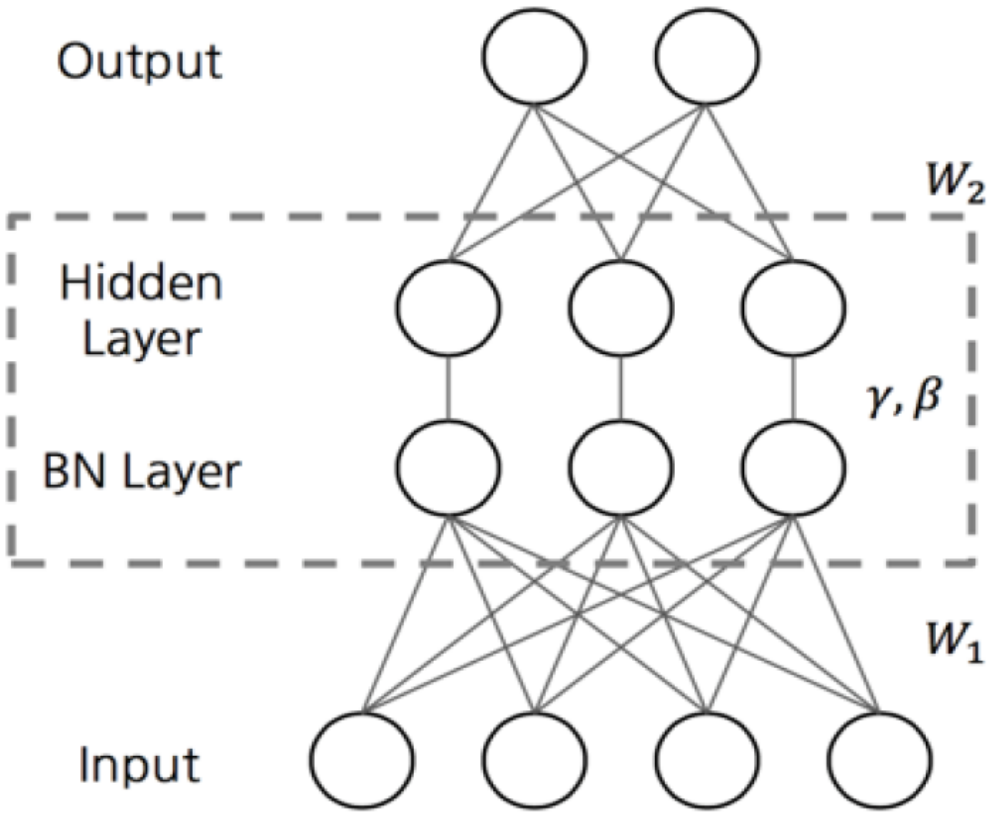
- Reducing internal covariate shift, improving training
- Scaling features similarly to prevent/reduce bias in the network
- Speeding up optimization by preventing weight explosion
- Reducing overfitting by aiding regularization

Types of normalization: Batch, Layer, Instance, Group normalization.

Batch Normalization

- The most common form of normalization in deep learning.
- Scales the inputs to a layer to a common value for every mini-batch during training.
- Stabilizes learning and significantly reduces the number of epochs needed, so the network trains faster.
- Computes the mean and variance of each feature in the mini-batch, subtracts the mean, and divides by the standard deviation.
- Adds an operation just before each layer's activation function: zero-center and normalize, then **scale and shift** using two new learnable parameters per layer.

Batch Normalization – Network Architecture



Batch Normalization: Equations

$$\mu_B = \frac{1}{m}\sum_{i=1}^{m} x_i \qquad \text{mini-batch mean}$$

$$\sigma_B^2 = \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_B)^2 \qquad \text{mini-batch variance}$$

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} \qquad \text{normalize}$$

$$y_i = \gamma\hat{x}_i + \beta \equiv BN_{\gamma,\beta}(x_i) \qquad \text{scale and shift}$$

- μ_B: empirical mean over the mini-batch B; σ_B: empirical standard deviation over B
- m: number of instances in the mini-batch
- γ: scaling parameter; β: shifting (offset) parameter, both learned per layer
- ϵ: small smoothing term avoiding division by zero (typically 10^{-5})

Batch Normalization: At Test Time

- At test time there is no mini-batch, so the whole training set's mean and standard deviation are used instead.
- These are typically computed efficiently during training via a **moving average**.
- Four parameters are learned per batch-normalized layer: γ (scale), β (offset), μ (mean), σ (standard deviation).
- Adds some model complexity and a runtime penalty (extra computation per layer at inference), but removes the need to separately normalize the input data.

Batch Normalization – Equations (Original Slide)

$$\mu_B = \frac{1}{m}\sum_{i=1}^{m} x_i \quad \rightarrow \text{mini-batch mean}$$

$$\sigma^2{}_B = \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_B)^2 \quad \rightarrow \text{mini-batch variance}$$

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma^2{}_B + \epsilon}} \quad \rightarrow \text{normalize}$$

$$y_i = \gamma\hat{x}_i + \beta \equiv BN_{y,b}(x_i) \quad \rightarrow \text{scale and shift}$$

Learning Curve

- Both the learning curve and validation curve plot training and validation scores on the y-axis.
- A **learning curve** plots score against the **number of training samples**.
- It helps determine whether an estimator would benefit from more data, or whether the model is too simple (biased).

Reading a Learning Curve

High Bias (Underfitting)

- Training and validation performance both "flat-line" at a low level.
- More data does not help – the model isn't finding the pattern.

High Variance (Overfitting)

- Large gap between training and validation performance.
- More data may not help much once validation performance plateaus.

Ideal case: Validation performance rises to meet training performance at a high level, with only a small gap between them (little to no overfitting).

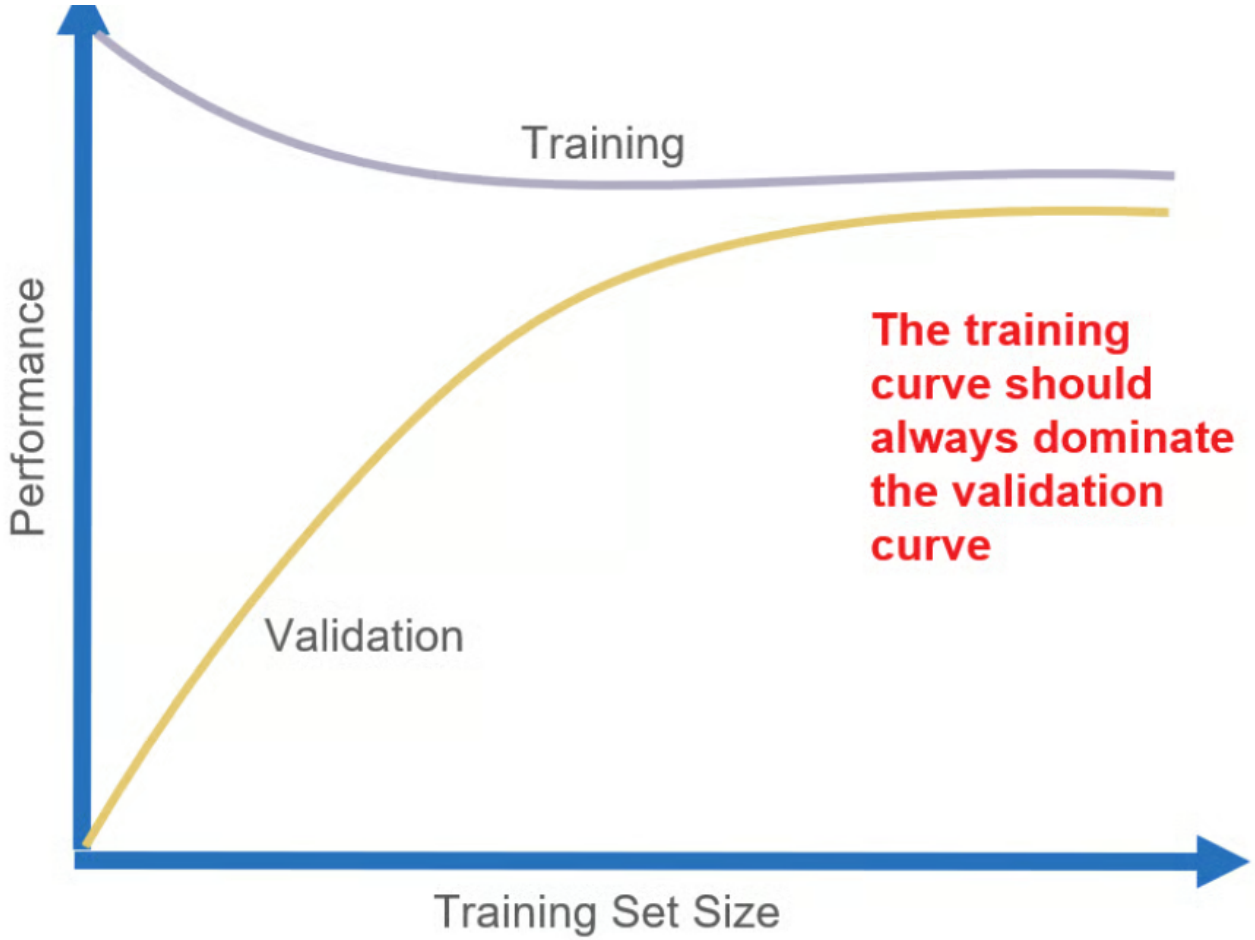
Learning Curve – High Bias



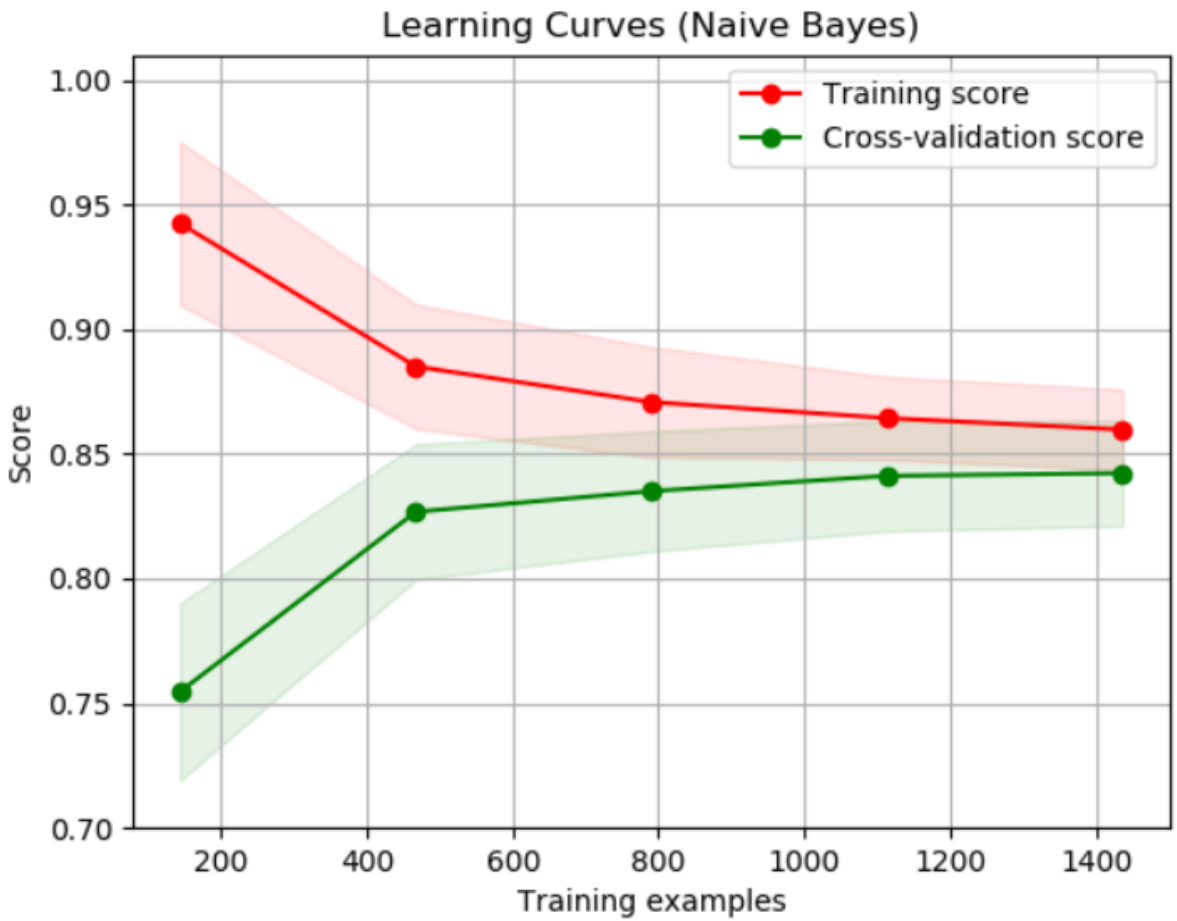
Learning Curve – High Variance (Overfitting)



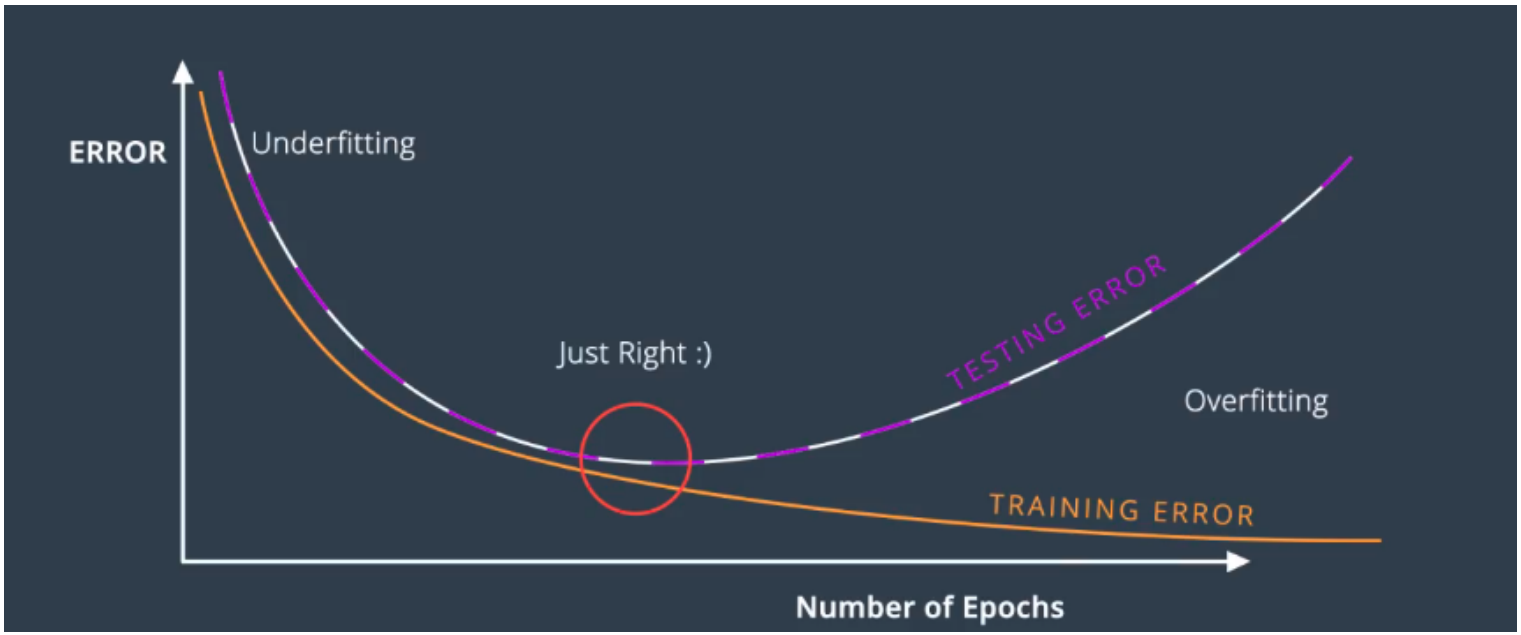
Learning Curve – Ideal Case



Learning Curve – Ideal Case (contd.)



Training Curve: Bias–Variance vs. Epochs



Key Observations in Learning Curves

- **Underfitting**: both training and validation errors are high and decrease very slowly – model isn't complex enough, or isn't trained long enough.
- **Overfitting**: training error is very low but validation error is high (or increasing) – model is too tuned to the training data.
- **Good fit**: both training and validation errors decrease and converge to a low value.

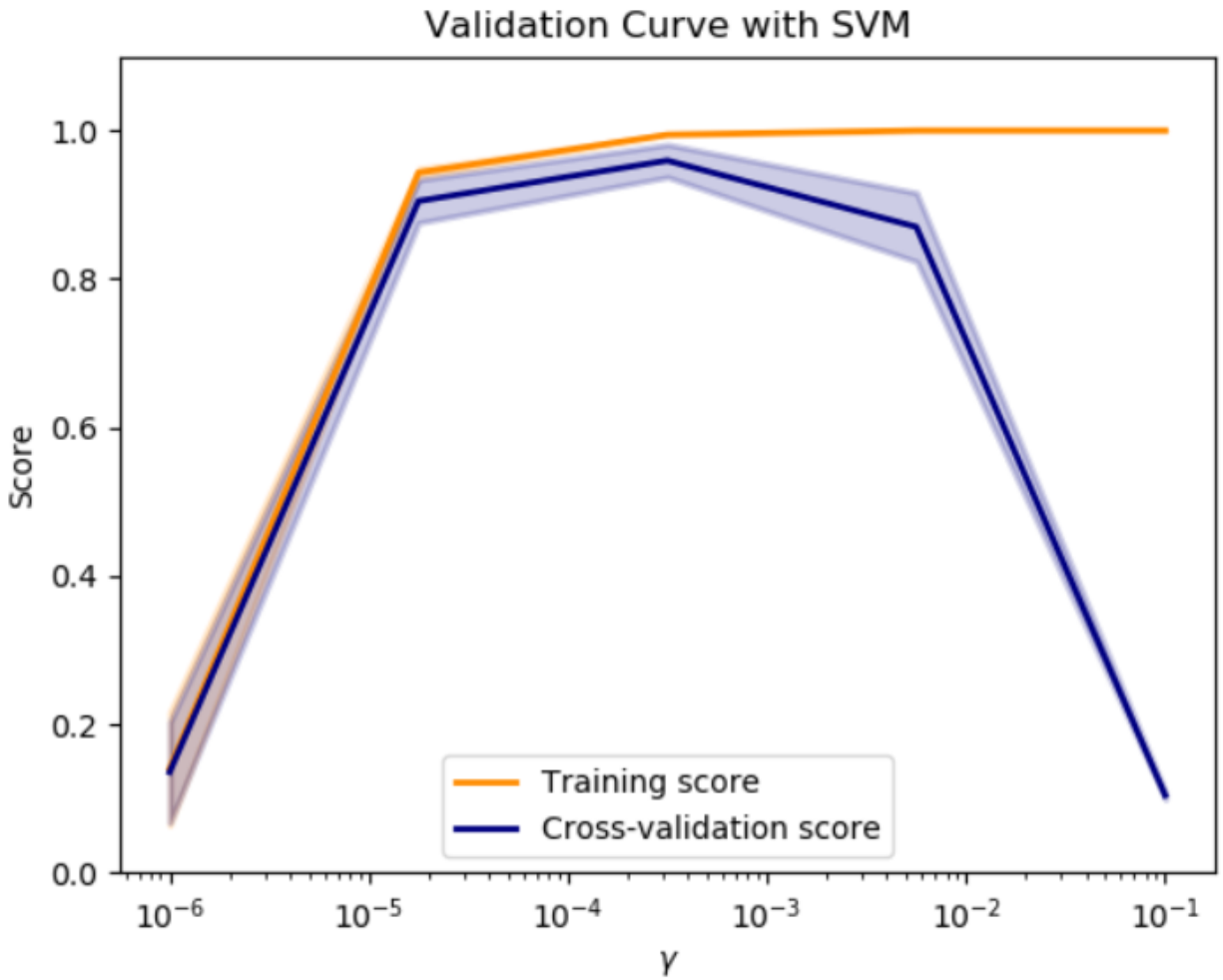
Validation Curve

- Evaluates model performance across different values of a **single hyperparameter** (e.g., regularization strength, learning rate, number of layers).
- Shows how training and validation metrics change as that hyperparameter varies.

Example: SVM gamma parameter

A too-low gamma restricts the model too much (both scores low). A too-high gamma causes overfitting (high training score, low validation score). The optimum lies where the curves stay close together.

Validation Curve – Diagram



Key Observations in Validation Curves

- **High Bias (Underfitting)**: both training and validation scores are low – the model is too simple.
- **High Variance (Overfitting)**: training score is high but validation score is low.
- **Optimal model**: both scores are close together – a good bias–variance tradeoff.

Note

A validation curve is used to *evaluate* an existing model against hyperparameters – it is not meant to be used to tune the model, or the model risks being biased toward the data it was tuned against.

Cross-Validation

- We cannot assume a model trained on the training data alone will work accurately on real/unseen data.
- **Cross-validation**: train the model on a subset of the dataset, then evaluate on the complementary subset.

Steps

1. Reserve a portion of the dataset.
2. Train the model on the rest.
3. Test the model on the reserved portion.

Methods of Cross-Validation

- **Validation (hold-out)**: Train on 50% of data, test on the other 50%. Drawback: important information in the untrained half is never used, leading to higher bias.
- **LOOCV** (Leave-One-Out Cross-Validation): Train on the whole dataset except one data point, iterating over every point.
  - \+ Low bias (uses all data points)
  - – High variance (tested against a single point); sensitive to outliers; computationally expensive
- **K-Fold Cross-Validation**: Split the dataset into k folds; train on $k-1$ folds and validate on the remaining fold, iterating k times.

K-Fold Cross-Validation: Advantages

vs. a single train/test split:

1. Runs k times faster than LOOCV (which repeats the split once per data point).
2. Simpler to examine detailed results of the testing process.

vs. no cross-validation at all:

1. More accurate estimate of out-of-sample accuracy.
2. More efficient use of data – every observation is used for both training and testing.

Data Visualization

- The graphical representation of information – charts, graphs, diagrams, pictures – can also serve as a reporting tool.
- Data visualization is an umbrella term for visualizing data through charts, graphs, and maps, in an accessible and easy-to-understand manner.
- Fundamental for understanding trends, uncovering patterns, and tracking outliers.

Common types

Bar graphs, line graphs, pie charts, scatter plots, histograms.

Python libraries

Matplotlib, Seaborn, Bokeh, Plotly.

Matplotlib

- An easy-to-use, low-level data visualization library built on NumPy arrays, offering scatter plots, line plots, histograms, and a lot of flexibility.
- `matplotlib.pyplot` provides an implicit, MATLAB-like interface for interactive and programmatic plotting.

```
import pandas as pd
import matplotlib.pyplot as plt

data = pd.read_csv("tips.csv")
plt.scatter(data['day'], data['tip'])
plt.title("Scatter Plot")
plt.xlabel('Day'); plt.ylabel('Tip')
plt.show()
```

Seaborn

- A high-level interface built on top of Matplotlib, offering more attractive default styles and color palettes.
- Key plotting functions: `scatterplot()`, `lineplot()`, `barplot()`, `histplot()` – each takes a `data` argument.

```
import seaborn as sns
sns.barplot(x='day', y='tip', data=data, hue='sex')
sns.histplot(x='total_bill', data=data, kde=True, hue='sex')
```

Bokeh and Plotly

Bokeh

Known for interactive charts; renders plots using HTML and JavaScript for elegant, concise, high-level interactivity in modern web browsers.

Plotly

Offers hover-tool capabilities to detect outliers/anomalies among data points, extensive customization, and visually attractive graphs.

t-SNE

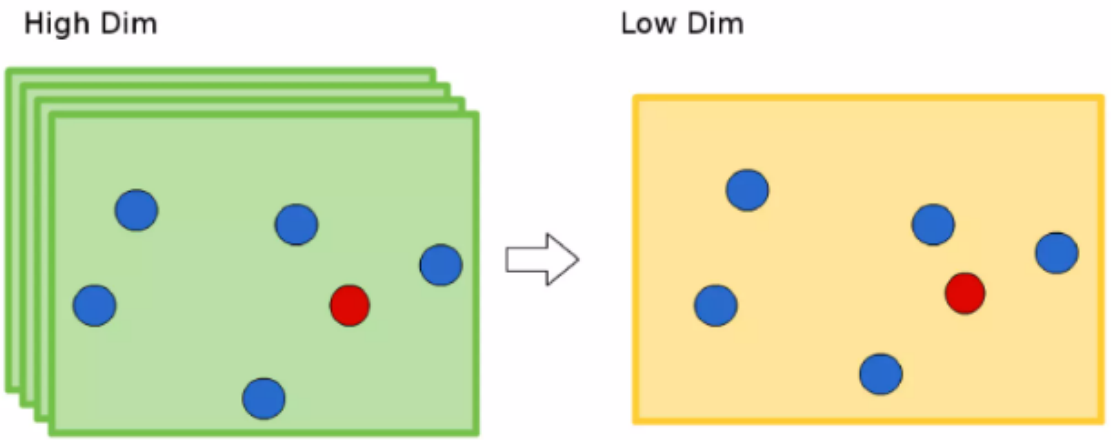
- **t-SNE**: t-Distributed Stochastic Neighbor Embedding – a **nonlinear** dimensionality reduction technique.
- Introduced in 2008 by Laurens van der Maaten and Geoffrey Hinton.
- Works well even on large datasets, and has become an industry standard.

Applications

Bioinformatics, cancer detection and disease diagnosis, natural language processing, and various deep-learning image-recognition tasks.

t-SNE – Visualization Example

Preserve the neighborhood



How t-SNE Works

- Goal: project multi-dimensional points into 2D/3D so that points close together in high-dimensional space stay close in the projection (and far points stay far).
- Builds a probability distribution over pairwise distances in the high-dimensional space, then optimizes a low-dimensional space with similar pairwise relationships.
- An optimization problem solved via **stochastic gradient descent**, minimizing **Kullback–Leibler (KL) divergence** between the two distributions.

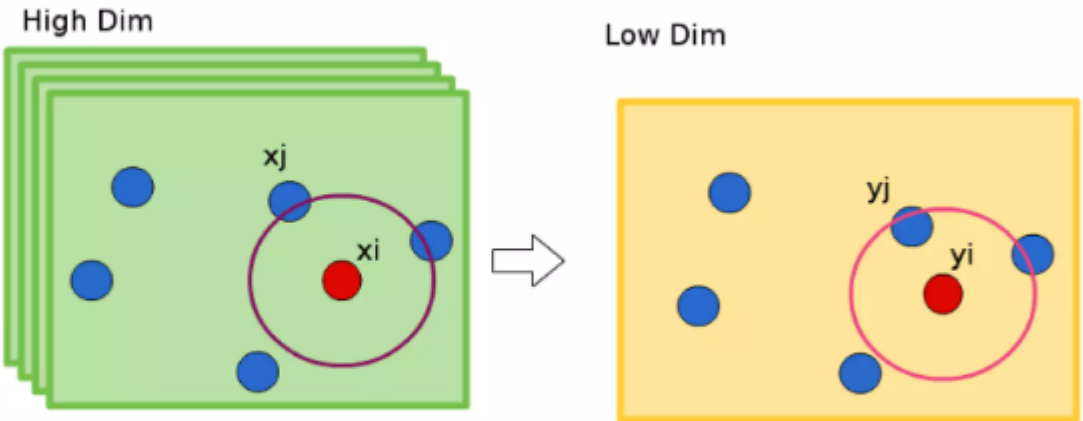
t-SNE: Step by Step

1. Compute pairwise similarity between all points in high-dimensional space using a Gaussian kernel – nearby points get higher probability of being picked as neighbors than far-apart points.
2. Map the high-dimensional points to a lower-dimensional space while trying to preserve those pairwise similarities.
3. Minimize the KL divergence between the high- and low-dimensional distributions using gradient descent.

The result reveals clusters and sub-clusters of similar points, useful for visualizing structure in high-dimensional data.

t-SNE – Step-by-Step Diagram

Measure pairwise similarities between high-dimensional and low-dimensonal objects



$$p_{j|i} = \frac{exp(-||x_i - x_j||^2/2\sigma_i^2)}{\sum_{k \neq i} exp(-||x_i - x_k||^2/2\sigma_i^2)}$$

Stochastic Neighbor Embedding

Converting the high-dimensional Euclidean distances into conditional probabilities that represent similarities

- Similarity of datapoints in High Dimension

$$p_{j|i} = \frac{exp(-||x_i - x_j||^2/2\sigma_i^2)}{\sum_{k \neq i} exp(-||x_i - x_k||^2/2\sigma_i^2)}$$

- Similarity of datapoints in Low Dimension

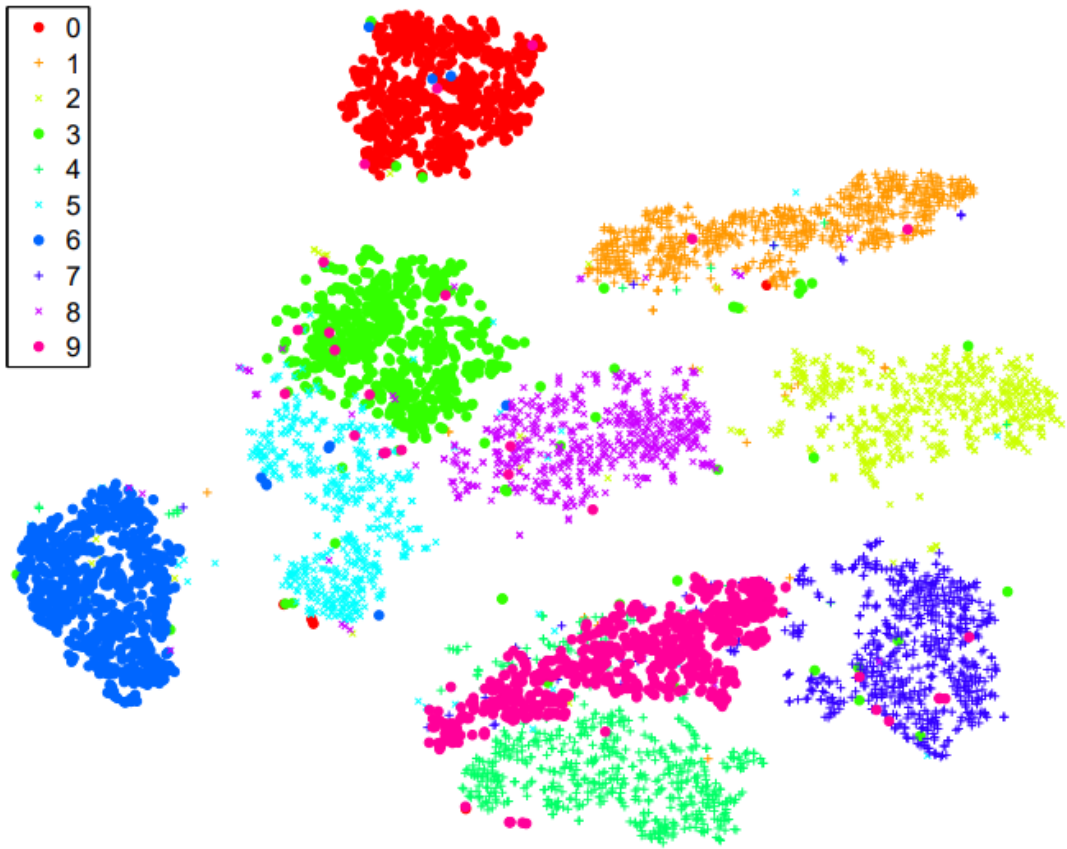
$$q_{j|i} = \frac{exp(-||y_i - y_j||^2)}{\sum_{k \neq i} exp(-||y_i - y_k||^2)}$$

- Cost function

$$C = \sum_i KL(P_i||Q_i) = \sum_i \sum_j p_{j|i} log \frac{p_{j|i}}{q_{j|i}}$$

Minimize the cost function using gradient descent

t-SNE – Additional Example (contd.)



(a) Visualization by t-SNE.

TensorFlow

- A free, open-source software library for machine learning and artificial intelligence, with a particular focus on training and inference of deep neural networks.
- Developed by the **Google Brain** team, originally for internal Google research and production; open-sourced in **2015** (first stable release in 2017).
- Supports Python (primary), as well as JavaScript, C++, and Java; runs on 64-bit Linux, macOS, Windows, Android, and iOS.
- **Tensor**: multi-dimensional array. **Flow**: computation based on data-flow graphs.
- TensorFlow's Keras-based APIs let users build their own machine learning models.

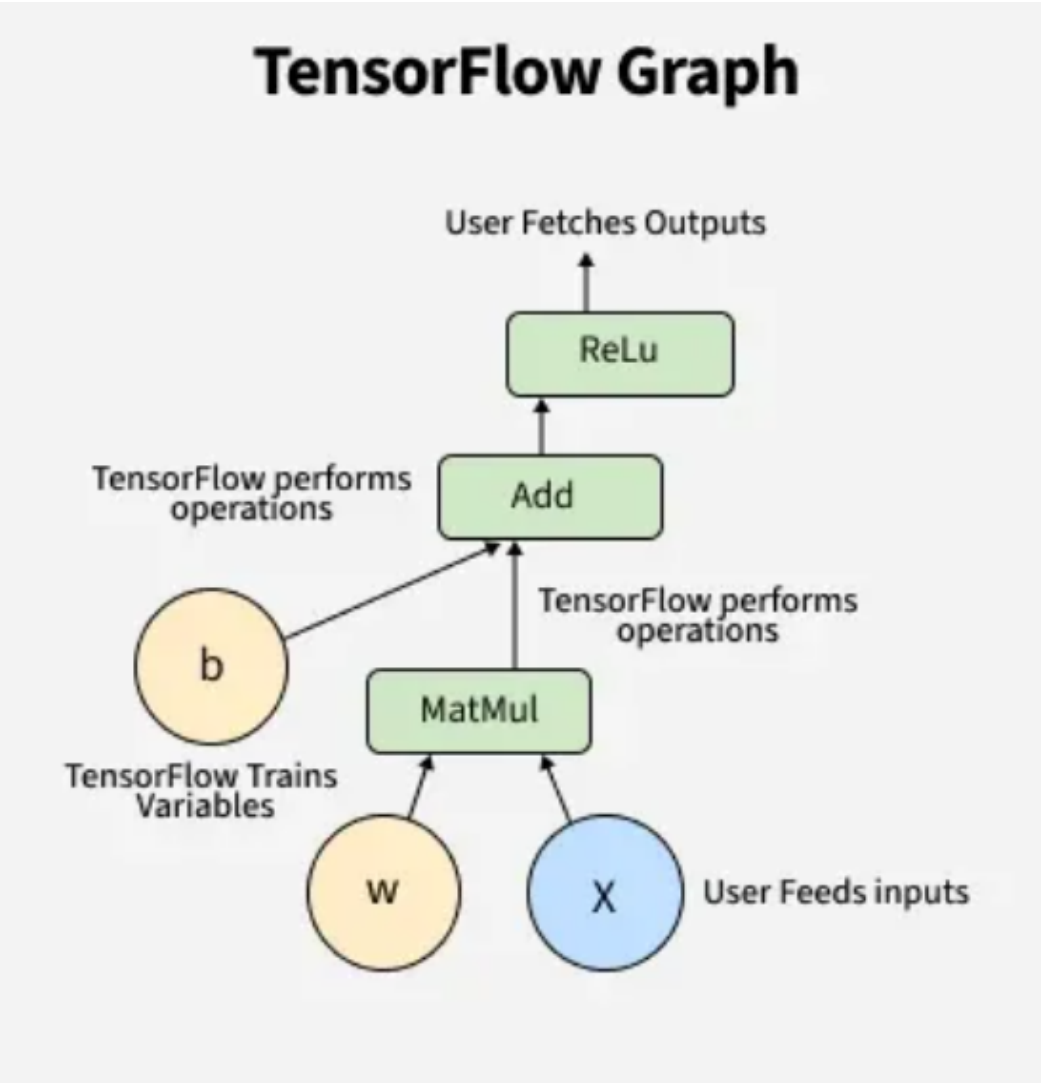
Key Concepts: Graphs and Nodes

- TensorFlow constructs and executes **dataflow graphs**, representing a series of computations.
- Each **node** (op) represents an operation (addition, multiplication, matrix multiplication, activation functions, ...), consuming tensors as input and producing tensors as output.
- **Edges** represent the flow of tensors between operations.
- The graph outlines operations and their connections, but does not itself display values – only the flow of data.

Advantages of the graph framework

Runs across multiple CPUs/GPUs and even mobile OSes; portable (can be saved and executed later); computations are performed entirely by connecting tensors together.

TensorFlow Graph – Neuron Example



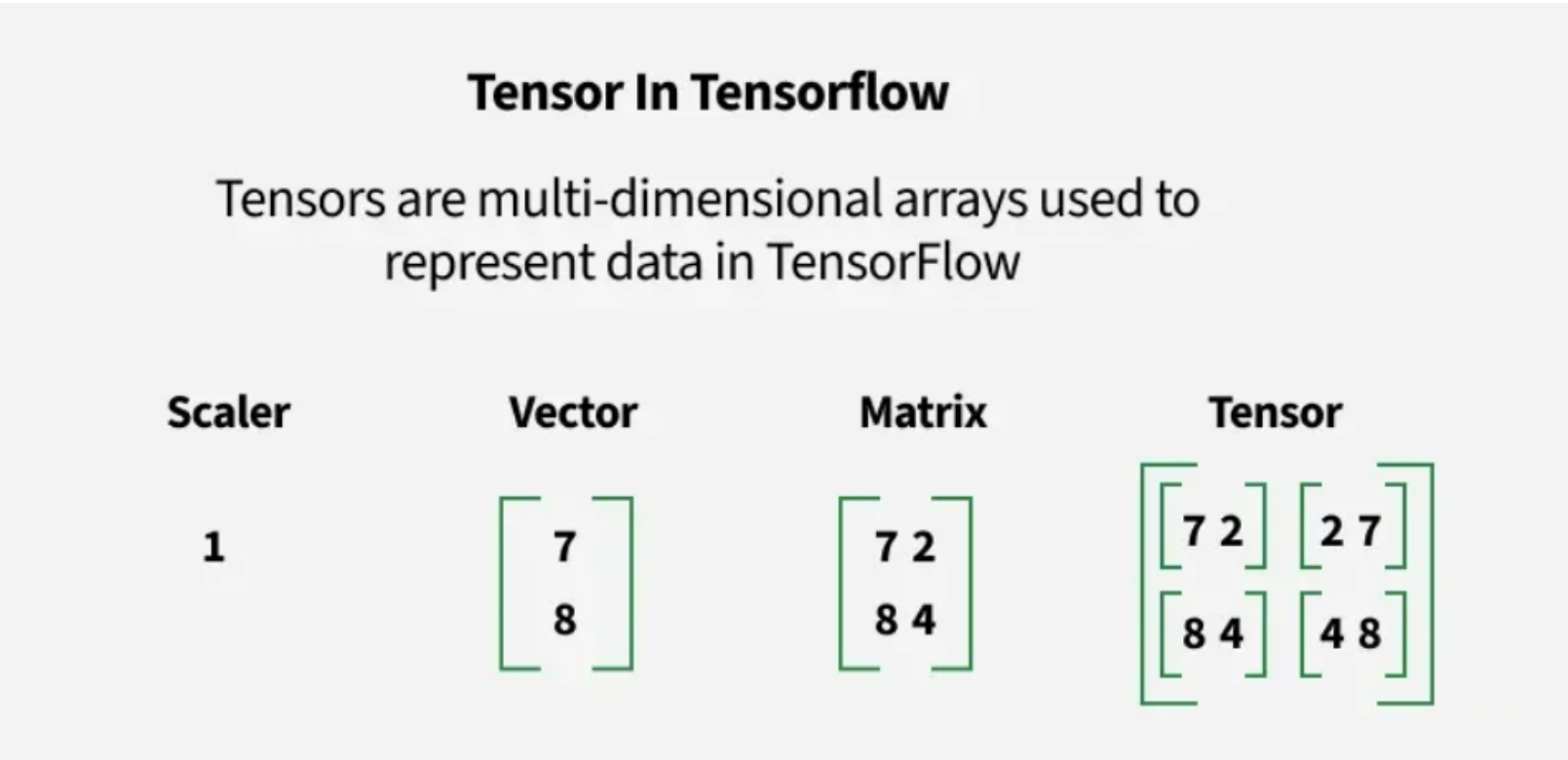
Tensors

- Tensors are the fundamental data structure in TensorFlow – multi-dimensional arrays or lists.
- All values in a tensor share the same data type, with a known (or partially known) shape.
- A tensor can originate from input data or from the result of a computation.

Dimensions of a Tensor

- Scalar – 0-dimensional array (a single number)
- Vector – 1-dimensional array
- Matrix – 2-dimensional array
- 3D Tensor – 3-dimensional array
- N-D Tensor – N-dimensional array

Tensor Dimensions – Diagram



Tensor Parameters

Tensors are identified by three parameters:

- **Rank**: the number of dimensions of the tensor (scalar = 0, vector = 1, matrix = 2, ...).
- **Shape**: the number of rows and columns (and higher dimensions) of the tensor.
- **Type**: the data type of the tensor's elements (e.g. `float32`, `int32`, `string`).

A tensor object also records the **device** it resides on (CPU, GPU, ...).

Creating Tensors

```
import tensorflow as tf

scalar_tensor = tf.constant(5)
vector_tensor = tf.constant([1, 2, 3, 4])
matrix_tensor = tf.constant([[1, 2], [3, 4]])
tensor_3d = tf.constant([[[1, 2], [3, 4]],
                         [[5, 6], [7, 8]]])
zeros_tensor = tf.zeros([3, 3])
ones_tensor  = tf.ones([2, 2])
```

Constants hold fixed values, while **Variables** hold model parameters (e.g. weights, biases) that are optimized during training and must be initialized before use.

TensorFlow 1.x vs. 2.x

TensorFlow 1.x

- Explicitly define a computation **graph**, then create a **session** to run it.
- "Lazy execution": operations are defined first, but only run when a session executes them.

TensorFlow 2.x

- **Eager execution** is enabled by default.
- Computations run immediately – no explicit session required.

TensorFlow 1.x: Building a Graph

```python
import tensorflow as tf

# Define a graph
graph = tf.Graph()

with graph.as_default():
    a = tf.constant(5)
    b = tf.constant(6)
    c = a * b

# Create a session to run the graph
with tf.compat.v1.Session(graph=graph) as sess:
    result = sess.run(c)
    print(result)
```

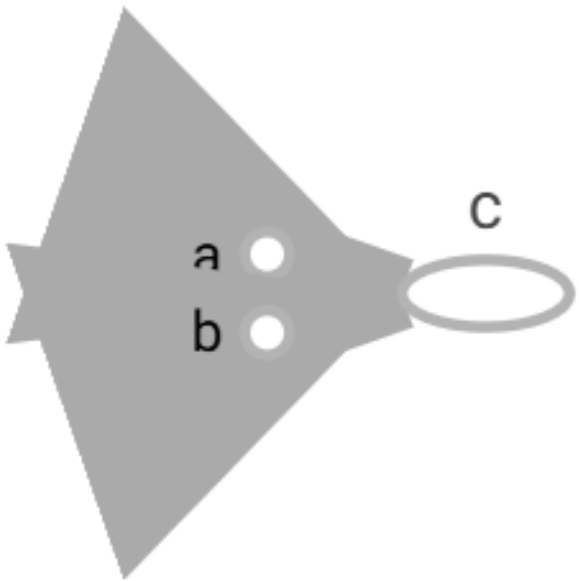
TensorFlow 2.x: Eager Execution – Diagram

```python
import tensorflow as tf

# No need for graph or session, eager execution runs immediately
a = tf.constant(5)
b = tf.constant(6)
c = a * b

print(c)
```

TensorFlow 2.x: Eager Execution – Graph View



TensorFlow Algorithms (`tf.estimator`)

- Linear regression: `tf.estimator.LinearRegressor`
- Classification: `tf.estimator.LinearClassifier`
- Deep learning classification: `tf.estimator.DNNClassifier`
- Wide and deep learning: `tf.estimator.DNNLinearCombinedClassifier`
- Boosted tree regression: `tf.estimator.BoostedTreesRegressor`
- Boosted tree classification: `tf.estimator.BoostedTreesClassifier`

Keras

- Keras is the **high-level API** of the TensorFlow platform, focused on modern deep learning.
- Covers every step of the ML workflow: data processing, hyperparameter tuning, and deployment.
- Can run on a TPU Pod or large clusters of GPUs; models can be exported to run in the browser, on mobile devices, or served via a web API.

Keras: Design Goals

Keras reduces cognitive load by:

- Offering simple, consistent interfaces.
- Minimizing the number of actions required for common use cases.
- Providing clear, actionable error messages.
- Following **progressive disclosure of complexity** – easy to start, with advanced workflows learned as you go.
- Helping developers write concise, readable code.

Keras: Layers and Models

- Core data structures: **layers** and **models**.
- A **layer** is a simple input/output transformation.
- A **model** is a directed acyclic graph (DAG) of layers – an object that groups layers together and can be trained on data.
- The simplest model type is `Sequential` – a linear stack of layers.
- For more complex architectures: the **Functional API** (arbitrary graphs of layers), or **subclassing** to write models from scratch.

Keras: Training and Evaluation

`tf.keras.Model` provides built-in training and evaluation methods:

- `Model.fit` – trains the model for a fixed number of epochs.
- `Model.predict` – generates output predictions for input samples.
- `Model.evaluate` – returns loss and metric values, as configured via `Model.compile`.

TensorFlow–Keras: Building a Neural Network Model

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense,Activation,Flatten,Dropout
import random
import tensorflow as tf
from tensorflow.keras.optimizers import SGD
```

```
random.seed(42)
np.random.seed(42)
tf.random.set_seed(42)
dnnModel=Sequential()
dnnModel.add(Dense(256,activation="relu",input_shape=(50176,)))
dnnModel.add(Dense(256,activation="relu"))
dnnModel.add(Dense(128,activation="relu"))
dnnModel.add(Dense(64,activation="relu"))
dnnModel.add(Dropout(0.5))
dnnModel.add(Dense(3,activation="softmax"))
dnnModel.summary()
```

TensorFlow–Keras: Compiling and Training

```
testLoss, testAccuracy = dnnModel.evaluate(X_test,y_test)
print("Test Accuracy =", testAccuracy)
```

TensorFlow–Keras: Fitting the Model

```
dnnModel.compile(optimizer='sgd',loss="categorical_crossentropy",metrics=["accuracy"])
```

```
history =dnnModel.fit(X_train,y_train,epochs=150,batch_size=32,verbose=1,validation_split=0.1)
```

TensorFlow–Keras: Prediction

```
pred = dnnModel.predict(img)
```

Thank You